

새 기기 작업 명세 — 2026-08-22

이 문서 하나만 읽고 시작할 수 있게 썼습니다. 대화 맥락을 옮기려 하지 마세요. 세션은 서로의 대화를 모릅니다.

새 Claude 세션에 붙여 넣을 첫 줄:

CLAUDE.md, 21_ZIF69_MTV/NEW_MACHINE_20260822.md 읽고
bash bgstate.sh 돌린 다음 현황 보고해

0. 30초 요약

MTV-ZIF 기반 CO₂ 포집 소재를 분자 시뮬레이션으로 설계·평가하는 학부 연구입니다. 실험은 없고 전부 계산입니다.

- **모체**: ZIF-69 (gme 위상), 단위셀 600원자, 치환 가능한 -Cl 자리 **24개**
- **주력 치환기**: -SO₃H (술포산). 치환율을 24분의 정수로 올라가며 비교
- **핵심 질문**: *습윤 배가스에서 어느 치환율이 가장 유리한가*
- **답(현재)**: **58.3% (14/24)** — 습윤 TSA 작업 용량 0.8328±0.0252 mol/kg, 무치환 대비 +77%, 차순위와 2.10σ
- **긴장의 정체**: CO₂를 세계 잡는 자리(-OH 수소결합 공여)가 **물도 세계 잡습니다**. 그래서 최대 치환이 최선이 아닙니다

이 기기가 할 일은 그 승자 조성의 수분 관문을 채우는 것입니다(4절).

1. 이 프로젝트가 반복해서 데인 것 — 먼저 읽으세요

11일 동안 근본 결함 4건이 났고 전부 같은 유형입니다: 실패가 결과처럼 보이는 것.

언제	무엇	대가
08-04	주기 경계 언랩 버그	10일 뒤 발견, 계산 전량 폐기
08-14	치환기 부착 원자 고정 인덱스	몇 주 묵음, v1 결과 전량 폐기
08-15	모체 기하 미최적화 · 조성 개수 · 힘장 표기	v3 전면 재구축
08-18	.gitignore 한 줄이 관문을 껴고 "30/30 통과"로 찍힘	관문이 무의미했음

그래서 규율이 있습니다 — 나쁜 결과를 보면 검사기부터 의심하고, 좋은 결과를 보면 기준을 의심하세요. 화면에 찍힌 "통과"가 통과가 아닐 수 있습니다.

2. 절대 바꾸면 안 되는 값

바꾸는 순간 기존 결과와 합칠 수 없습니다. 성능이 아쉬워도 바꾸지 말고 물어보세요.

항목	값
사이클	초기화 5,000 + 생산 15,000

항목	값
골격 힘장	UFF_MOF
CO ₂ 모델	García-Sánchez 2009 — ϵ 29.933 K, σ 2.745 Å, q_C +0.6512
물 모델	TIP5P-Ew (5자리)
전하	PACMAN DDEC6 (CIF에 포함)
정전기	Ewald 1e-6
컷오프	12 Å
단위셀	2×2×2
흡착 조건	CO ₂ 0.15 bar, 298 K

경로가 molecules/TraPPE/CO2.def **지만 TraPPE가 아닙니다**. 그 줄은 분자 기하만 고릅니다. LJ와 전하는 forcefield/UFF_MOF/ 의 두 파일에서 옵니다. 문서·장표·보고서에 "TraPPE"라고 적으면 틀립니다. 물도 같습니다 — 배포본 TraPPE/water.def 는 3자리, 우리 것은 19_WaterCompetition/water.def 의 5자리입니다. 바뀌치면 조용히 다른 물로 계산되고 결과는 멀쩡해 보입니다.

결과를 읽는 규율

- 1 σ 를 병기하고, 차이가 1.5 σ 미만이면 순위를 매기지 않습니다. 1.5 σ 미만은 "같다"가 아니라 "모른다"입니다
- 작업 용량은 두 로딩의 차이로 오차가 더해집니다: $\sigma_{wc} = \sqrt{(\sigma_{ads}^2 + \sigma_{des}^2)}$
- 판정 기준은 계산을 돌리기 전에 등록합니다. 결과를 보고 고치지 않습니다
- v1/v2/v3는 서로 다른 구조에서 나왔습니다. 세대를 넘나들며 인용 금지

3. 환경 구축 — 이 순서 그대로

3-1. 저장소

```
git clone https://github.com/mangwon3-commits/Czero_mtv_mof.git ~/mof_project
cd ~/mof_project
```

비공개 저장소입니다. 접근이 안 되면 사용자에게 초대를 요청하세요.

clone 하나로 계산에 필요한 것이 전부 들어옵니다 — 힘장 트리(00_Migration/raspa_share/), 5자리 물 정의(19_WaterCompetition/water.def), 전하가 붙은 v3 구조 38종(21_ZIF69_MTV/charged_v3/), 러너 전부. 따로 받아야 하는 파일은 없습니다.

3-2. 파이썬 환경

이 작업에 필요한 것은 세 개뿐입니다. 다른 env(coremof_tools, lammops_mof, spectra)는 이 일감에 필요 없습니다.

```
conda create -n czeromof -c conda-forge python=3.10 \
raspa2=2.0.50 ase=3.29 numpy -y
```

3-3. RASPA 힘장 설치 — 여기서 가장 많이 틀립니다

RASPA는 \$RASPA_DIR/share/raspa/ 를 읽습니다. 배포본 기본 파일을 그대로 쓰면 우리 값이 안 나옵니다. 저장소에 우리 트리가 통째로 들어 있습니다.

```
export RASPA_DIR=$HOME/RASPA/simulations
mkdir -p "$RASPA_DIR/share/raspa"
cp -r ~/mof_project/00_Migration/raspa_share/raspa/* "$RASPA_DIR/share/raspa/"
```

소스 빌드 RASPA 주의(08-20에 실제로 걸림): 소스 트리에는 molecules/TraPPE/ 폴더가 **아예** 없습니다. CO₂/N₂ 정의가 \$RASPA_DIR/share/raspa/molecules/TraPPE/{CO₂,N₂}.def 에 있어야 하고, molecules/ 바로 아래 두면 **절대 읽히지 않습니다**. conda의 raspa2를 쓰면 이 문제는 없습니다.

3-4. 설치 검증 — 반드시 통과시키고 넘어가세요

```
cd ~/mof_project/21_ZIF69_MTV
export RASPA_DIR=$HOME/RASPA/simulations

# ① 힘장이 우리 것인가 (셋 다 나와야 함)
grep -E "^C_co2" "$RASPA_DIR/share/raspa/forcefield/UFF_MOF/pseudo_atoms.def"
# -> q = 0.6512 여야 합니다
grep -E "C_co2|N_n2" \
"$RASPA_DIR/share/raspa/forcefield/UFF_MOF/force_field_mixing_rules.def"
# -> C_co2 29.933 2.745 / N_n2 38.298 3.306

# ② 물이 5자리인가
awk 'NF>4 && ($2=="Ow"||$2=="Hw"||$2=="Lw")' \
../19_WaterCompetition/water.def | wc -l
# -> 5

# ③ 러너가 보는 환경에서 의존이 잡히는가 (PATH 를 비우고 확인)
env -i HOME=$HOME PATH=/usr/bin:/bin \
~/miniconda3/envs/czeromof/bin/python -c \
"import sys; sys.path.insert(0, '.'); import run_water as rw, os; \
print(rw.SIMULATE, os.path.exists(rw.SIMULATE))"
# -> 경로와 True
```

which simulate **만 쳐 보고 "있다"고 판단하지 마세요**. 그건 당신 셸의 PATH를 보는 것이고, 무인 실행(setsid nohup) 러너가 보는 PATH가 아닙니다. 08-19에 이 차이로 배정 하나가 실행 불가였습니다.

3-5. WSL이라면

- WSL2는 기본으로 **호스트 RAM의 절반만** 씁니다. 16 GB 기기면 WSL은 8 GB
- RASPA simulate 한 건은 **약 471 MB**. 16건이어도 7.5 GB — 메모리가 아니라 **코어가 병목**입니다
- **Zeo++ network 는 v3 구조에서 한 건에 약 9.5 GB**입니다. RASPA가 돌 때 같이 띄우지 마세요 — 08-12에 이 조합이 OOM을 내고 dbus까지 죽여 WSL이 통째로 먹통이 되었습니다. 이번 일감에는 Zeo++가 없으니 그냥 띄우지 마세요

4. 이 기기의 일감 — 조성 격자 4종의 수분 경쟁

4-1. 왜 이것이 비어 있나

08-20에 외부 16코어가 수분 경쟁 v3를 돌렸는데 대상이 base / 025 / 050 / 075 / 100 다섯 중이었습니다. 그 뒤 50~75% 사이를 가르려고 격자 4종(58.3 / 62.5 / 66.7 / 87.5%)을 만들었고, 건조 흡착·구조 안정성·습윤 작업 용량까지 다 났는데 — **RH 스캔만 안 났습니다**.

그 결과, 지금 승자인 saIm0583에 대해 사전 등록된 수분 관문 (RH90 유지율 ≥ 80 유효 / 50~80 조건부 / <50 종료)을 한 번도 적용한 적이 없습니다.

RH90 한 점은 습윤 작업 용량의 흡착 조건에서 간접적으로 있습니다. 없는 것은 RH 25 / 50 이고, 그래서 물이 어느 습도에서 끼어드는지(온셋) 를 말할 수 없습니다. Part 7 12-3절이 "온셋 위치가 문헌에 같은 축이 없는 우리 데이터의 독자적 기여"라고 적은 바로 그 축입니다.

4-2. 무엇을 계산하나

구조 4종 $\times$ RH 0 / 25 / 50 / 90 = 16작업의 CO₂/H₂O 이원 GCMC.

태그	치환율	자리	왜
saIm0583	58.3%	14/24	현 승자. 최우선
saIm0625	62.5%	15/24	안정성 관문이 닫히는 칸
saIm0667	66.7%	16/24	그 위
saIm0875	87.5%	21/24	Q _{st} 문턱이 열리는 칸

물 분압은 298 K 기준입니다: RH0 = 0 Pa · RH25 = 792 · RH50 = 1,585 · **RH90 = 2,852 Pa**. CO₂는 0.15 bar 고정.

4-3. 명령

```
cd ~/mof_project/21_ZIF69_MTV
export RASPA_DIR=$HOME/RASPA/simulations
export PATH=$HOME/miniconda3/envs/czeromof/bin:$PATH

# <N> = 물리 코어 수 (논리 코어 아님). 아래 4-4 참조
WATER_V3_WORKERS=<N> setid nohup \
python -u run_water_v3grid.py > water_v3grid.log 2>&1 < /dev/null &
```

러너가 시작할 때 **힘장·물 정의·전하 CIF**를 스스로 검사하고, 5자리 물이 아니면 계산을 시작하지 않고 종료합니다.

완료 파일: v3_water_grid/water_results.json (16행)

중단돼도 같은 명령을 그대로 다시 치세요. 작업 단위 이어받기가 있어 완주분을 건너뛸니다. 08-17 정전 때 이 방식으로 19작업을 살렸습니다. **단, 이어받기는 출력 파일의 존재만 봅니다. 무슨 설정으로 만들어졌는지 검사하지 않습니다.** 설정을 건드렸다면 재개하지 말고 water_runs_v3grid/ 를 지우고 처음부터 도세요.

4-4. 워커 수 — 물리 코어 수로

```
lscpu | grep -E "^CPU\(\s\)|Core\(\s\)" | grep per socket|Thread"
python -c "import os; print('논리', os.cpu_count())"
```

하이퍼스레딩된 논리 코어 수를 넣지 마세요. RASPA는 코어를 꽉 채워 쓰기 때문에 논리 코어로 늘리면 오히려 느려집니다. 물리 8코어면 WATER_V3_WORKERS=8.

4-5. 예상 소요 시간

기준: 08-20에 외부 16코어가 20작업을 18.2시간에 완주(같은 러너, 같은 사이클 수, 종료코드 0).

물리 코어	예상
16코어	12 ~ 18시간 (16작업이 한 파도에 들어감)

물리 코어	예상
8코어	22 ~ 32시간 (두 파도)
4코어	45 ~ 60시간 (네 파도)

작업 하나의 길이가 최대 13배까지 차이 납니다(1.5~20.2시간). 그래서 파도 수가 늘면 시간이 선형보다 나쁘게 늘어납니다. 코어가 4개 이하면 사용자에게 알리고 대상을 saIm0583 하나(4작업)로 줄이는 편이 낫습니다 — 승자 조성 하나만으로도 관문은 채워집니다.

4-6. 진행 상황을 세는 법 (전부 틀렸던 방법들)

- 실행 폴더 개수로 세지 마세요. 결과를 뽑으면 폴더를 지웁니다. 끝난 것일수록 안 보입니다
- 출력 파일이 몇 시간째 그대로여도 멈춘 게 아닙니다. PrintEvery 가 생산 사이클 수와 같아 처음과 끝에만 씁니다
- pgrep -f 는 자기 명령줄에 매칭됩니다. pgrep -x simulate 를 쓰세요. 이걸로 자기 셸을 죽인 적이 있습니다
- CPU 0%인 드라이버는 대개 정상입니다(자식이 계산 중). 무리 전체가 0%일 때만 의심하세요
- 한 번에 보려면 bash bgstate.sh

진행 확인:

```
tail -20 water_v3grid.log
pgrep -x simulate | wc -l # 지금 도는 RASPA 건수
python -c "import json;print(len(json.load(open('v3_water_grid/water_results.json'))))"
```

5. 결과 반환

```
cd ~/mof_project/21_ZIF69_MTV
tar czf water_v3grid_$(hostname)_$(date +%m%d).tar.gz \
v3_water_grid/water_results.json water_v3grid.log
sha256sum water_v3grid_*.tar.gz > water_v3grid_$(hostname)_$(date +%m%d).sha256
```

이 두 파일을 사용자에게 전달하세요. v3_water/water_results.json(기존 5종)에 직접 합치지 마세요. 데스크탑이 교차 검증한 뒤 사람이 합칩니다 — 어느 기기가 낸 값인지 되돌릴 수 있어야 합니다.

결과가 나오면 무엇을 말할 수 있나

사전 등록된 기준을 그대로 적용합니다(결과를 보고 고치지 않습니다):

```
유지율 = RH90 CO2 로딩 ÷ RH0 CO2 로딩 × 100%
≥ 80% 유효
50 ~ 80% 조건부
< 50% 노선 종료
```

유지율만 적으면 결론이 거꾸로 읽힙니다. 비율이라 분모가 조성마다 크게 다릅니다. 표에 절대 로딩을 반드시 병기하세요.

기대되는 위치: 기존 5종에서 25~75% 구간이 76~80%의 고원이었고 100%에서 60.6%로 꺾였습니다. 격자 4종은 그 고원과 꺾이는 지점 사이를 채웁니다.

6. 하지 말아야 할 것

금지	이유
구조를 새로 만들기	검사 항목이 다릅니다(충돌·고아 원자·조성 개수). 08-14 결함이 정확히 그 자리에서 났습니다. 구조 생성은 사람이 붙어 있는 데스크탑에서만
rebuild_structures.py 실행	argparse가 없어 --help 만 줘도 즉시 structures_v2/ 를 통째로 덮어씁니다(08-19에 실제로 발생)
돌고 있는 bash 스크립트 편집	바이트 오프셋으로 읽어 엉뚱한 줄을 실행합니다. 파이썬은 편집해도 실행 중 프로세스엔 반영 안 되고 재기동할 때 터집니다
21_ZIF69_MTV/density_v2/ 삭제	VTK 밀도 격자가 그림의 원본이고 사본이 없습니다
결과 JSON · structures_v2/ · charged_v3/ · relax_v3/*_relaxed.cif 삭제	재생성 비용이 수십 시간입니다
사이클 수·힘장·컷오프 조정	기존 결과와 합칠 수 없게 됩니다
RISK_WORKERS 를 손으로 주기	스크립트가 /proc/meminfo 에서 계산합니다 (이번 일감엔 해당 없음)
대화에서만 존재하는 용어 만들기	08-17에 "티어 1~4"를 대화에서 만들어 여러 지시에 썼는데 저장소 어디에도 정의가 없었습니다. 새 용어를 만들면 쓰기 전에 파일로

실행 폴더(*_runs*/)는 결과 JSON이 있으면 지워도 됩니다. cleanup_for_compact.sh 가 관문을 걸어 검사합니다.

7. 지금 다른 기기가 무엇을 하고 있나

충돌을 피하려면 이 표를 보세요.

기기	작업	상태
데스크탑	sal0583 자리 배치 양상불 5실현 (이완→전하→건조 GCMC)	08-22 11:00 시작, 8~12시간
랩탑	(유휴)	—
외부 16코어	수분 경쟁 v3 5종	08-20 완료
새 기기(당신)	조성 격자 4종 수분 경쟁 16작업	이 문서

데스크탑의 양상불은 structures_v2/ZIF69_saIm0583e{1..5} 와 results_v3ens0583.json 을 건드립니다. 당신의 일감은 v3_water_grid/ 와 water_runs_v3grid/ 만 씁니다 — **겹치지 않습니다**.
각 단계 끝에서 커밋·푸시하므로 GitHub 커밋 목록으로 서로의 진행을 볼 수 있습니다. 대화로 상태를 주고받으려 하지 마세요.

8. 더 읽을 것 — 필요할 때만

궁금한 것	문서
작업 규약 전체	CLAUDE.md (루트)
용어 정의 (Q _{st} , RH90, 작업 용량, σ, *■■■■)	21_ZIF69_MTV/GLOSSARY.md

궁금한 것	문서
최종 판정과 근거	21_ZIF69_MTV/FINAL_VERDICT_20260821.md
승자 판정 규칙(사전 등록)	21_ZIF69_MTV/DECISION_RULE_20260820.md
기기가 바뀌면 터지는 것들	21_ZIF69_MTV/ENV_ASSUMPTIONS.md
문헌 대조	21_ZIF69_MTV/LITERATURE_20260819.md
v3 결과 해석	21_ZIF69_MTV/V3_RESULT_NOTE.md
양상불 시험 사전 등록	21_ZIF69_MTV/ENSEMBLE_0583_PROTOCOL.md

9. 막히면

추측해서 진행하지 말고 멈추고 물어보세요. 이 프로젝트에서 가장 비싼 실패는 전부 "그럴듯하게 완주한 계산"이었습니다. 계산이 안 도는 것은 싸게 고칩니다. 틀린 채로 도는 것은 몇 주 뒤에 발견됩니다.